AI CRITIQUE OF THE GENERATED TEST SUITE

Digital Wallet Transfer Limits & Account Tiers

Document Reviewed: *Fintech_Wallet_Transfer_Test_Suite.xlsx*

Reviewer: Michael Oloruntobi

Purpose: Evaluation of AI-generated test cases

Executive Summary

This report evaluates the quality of the AI-generated test suite for the Digital Wallet Transfer Limits & Account Tiers feature. The objective was to determine whether the generated test cases accurately reflect the requirements, identify incorrect or redundant scenarios, highlight hallucinated functionality, and assess the strengths and limitations of AI-assisted test design.

Overall, the AI produced a comprehensive baseline test suite with strong coverage of functional requirements, particularly around Boundary Value Analysis (BVA), Equivalence Partitioning (EP), and negative testing. The generated suite demonstrates AI's ability to rapidly produce large numbers of structured test cases with consistent formatting and broad functional coverage.

However, the review also revealed several weaknesses typical of Large Language Models (LLMs). The AI frequently assumed implementation details that were not present in the specification, produced duplicate tests covering identical business rules, and failed to identify several important ambiguities and high-risk scenarios that an experienced QA engineer would normally investigate.

After review, the majority of the generated test cases were considered suitable for retention with only minor modifications. Others require correction to remove implementation assumptions, while a smaller number should be removed because they duplicate existing coverage or verify behaviour not described in the requirements.

This exercise demonstrates that AI should be viewed as a test design accelerator rather than an autonomous test designer. Human review remains essential to validate assumptions, remove hallucinated behaviour, and ensure adequate risk-based coverage.

Review Methodology

Each generated test case was assessed against the original feature specification using the following criteria.

Requirement Traceability

Every test case was checked to determine whether it directly validates a stated business requirement.

Technical Accuracy

Expected results were reviewed to ensure they verify business behaviour rather than invent implementation details.

Coverage

The overall suite was evaluated to determine whether all significant functional requirements and business rules were exercised.

Redundancy

Duplicate and overlapping scenarios were identified to reduce maintenance effort while preserving functional coverage.

Risk Coverage

The suite was assessed for coverage of realistic business risks, edge cases and operational scenarios.

Overall Assessment

Evaluation Area	Rating	Comments
Functional Coverage	**Excellent**	Covers most stated business rules
Boundary Value Analysis	**Excellent**	Strong use of lower, upper and off-by-one boundaries
Equivalence Partitioning	**Very Good**	Appropriate valid and invalid partitions
Negative Testing	**Good**	Most invalid input scenarios covered
Specification Compliance	**Good**	Several implementation assumptions introduced
Risk-Based Coverage	**Fair**	Important operational scenarios omitted
Maintainability	**Fair**	Significant duplication of boundary tests

Test Cases Retained

Approximately 75% of the generated test suite should be retained because these cases directly validate explicit requirements contained within the specification.

1. Tier-Based Transfer Limits

The specification defines maximum single-transfer limits for each account tier. The AI correctly generated scenarios covering:

- Transfers below the limit
- Transfers exactly at the limit
- Transfers exceeding the limit
- Minimum valid transfer values
- Invalid zero-value transfers

These cases provide comprehensive Boundary Value Analysis and represent high-value regression tests.

Decision: Retain.

2. Daily Transfer Limits

The suite contains appropriate scenarios validating cumulative daily transfer limits. Examples include:

- Multiple transfers reaching the daily threshold
- Attempting a transfer that exceeds the daily allowance
- Successful transfers while remaining within the daily limit

These scenarios accurately reflect the business rules and should remain.

Decision: Retain.

3. Wallet and Bank Transfers

The requirements distinguish between transfers made to:

- Another wallet user
- A bank account

The generated tests ensure that transfer limits apply consistently to both transfer destinations.

Decision: Retain.

4. Boundary Value Analysis

The AI produced systematic boundary tests covering:

- Just below the limit
- Exactly at the limit
- Just above the limit

This represents one of the strongest aspects of the generated suite. These tests provide excellent regression coverage while minimising the likelihood of limit validation defects.

Decision: Retain.

Test Cases Requiring Correction

Several generated cases verify behaviour that cannot be derived from the specification. These tests should not be removed entirely but should instead be rewritten to validate business behaviour rather than implementation details.

Example 1

Generated Expectation: Submit button becomes disabled.

Issue: The specification contains no information regarding the application's user interface. Whether a button becomes disabled, hidden or remains enabled is an implementation decision.

Correct Expectation: The transfer request should be rejected because the amount violates the business rules.

Example 2

Generated Expectation: HTTP 400 Bad Request

Issue: The requirements never mention REST APIs, HTTP status codes, or response payloads. This behaviour has been invented by the AI.

Correct Expectation: The transfer should fail and an appropriate validation error should be returned.

Example 3

Generated Step: User enters transfer PIN.

Issue: Authentication is never mentioned within the specification. PIN verification may exist within the application but cannot be assumed from the provided requirements.

Decision: Rewrite as an implementation-independent business validation test.

Hallucinated Behaviour

The review identified multiple examples where the AI introduced functionality that does not exist within the specification. These hallucinations demonstrate a common limitation of LLM-generated test design.

UI Behaviour

Examples include:

- Disabled buttons
- Confirmation dialogs
- Progress indicators
- Remaining daily limit displays

None of these are described in the requirements.

API Behaviour

Examples include:

- HTTP status codes
- Error payloads
- Validation schemas
- Response objects

Again, these are implementation assumptions rather than requirement-derived expectations.

Exact Error Messages

Examples such as "Amount must be greater than zero" should not be verified unless explicitly defined. Instead, tests should verify that an appropriate validation message is displayed.

Redundant Test Cases

The generated suite contains unnecessary duplication. For example, several boundary tests verify exactly the same business rule using different identifiers. Typical duplication includes:

- Individual boundary tests
- Boundary matrix tests
- Combined equivalence partition tests

All verify the same outcome. Reducing these duplicates would improve maintainability without reducing coverage.

Similarly, multiple generic input validation tests verify identical behaviour for:

- Negative values
- Zero values
- Blank values

A smaller representative set would be sufficient.

Missing Test Scenarios

Although coverage is extensive, several important scenarios were not generated.

Concurrent Transfers

Two simultaneous transfers could individually pass validation while collectively exceeding the daily limit. This is a high-risk scenario in financial systems.

Failed Transfer Accounting

If a transfer fails after validation, does it count towards the daily limit? The specification is silent. This ambiguity should have been identified.

Reversed Transactions

If a transfer is reversed, is the customer's available daily allowance restored? No tests address this.

Tier Upgrade During the Same Day

If a customer upgrades from Tier 1 to Tier 2 after reaching their Tier 1 daily limit, are new limits immediately applied? No generated test explores this scenario.

Daily Reset Time

The specification never states whether reset occurs at midnight, on a rolling 24-hour window, or how time zones are handled. Rather than assuming behaviour, AI should have identified this ambiguity for clarification.

Currency Precision

No scenarios validate decimal precision, rounding behaviour, or invalid decimal formats. These are common financial defects.

Retry and Idempotency

Network failures often cause duplicate requests. The generated suite contains no scenarios validating duplicate submission handling or idempotency.

Strengths of AI Test Generation

This exercise clearly demonstrates several areas where AI provides significant value.

Rapid Test Generation

The AI produced a comprehensive suite within minutes. Creating an equivalent manual suite would require substantially more effort.

Strong Use of Standard Test Design Techniques

The generated tests make effective use of:

- Boundary Value Analysis
- Equivalence Partitioning
- Positive testing
- Negative testing

These represent established industry best practices.

Consistent Documentation

Every test follows a uniform structure, making the suite easy to read and maintain.

Broad Functional Coverage

The majority of explicit business rules were exercised. This makes AI an excellent starting point for regression suite development.

Weaknesses of AI Test Generation

Despite its strengths, the review highlights important limitations.

Assumption-Based Reasoning

Rather than remaining faithful to the specification, the AI frequently invented:

- APIs
- UI behaviour
- PIN verification
- Error messages

This reduces trust in generated artefacts.

Lack of Critical Thinking

Experienced QA engineers naturally question incomplete requirements. Examples include:

- When does the daily limit reset?
- Are failed transfers counted?
- What happens after KYC upgrades?
- Can transfers be reversed?

The AI instead assumed answers.

Limited Risk-Based Testing

The suite focuses heavily on straightforward validation while overlooking operational risks such as:

- Race conditions
- Concurrent transactions
- Fraud scenarios
- Recovery behaviour
- System resilience

These often represent the highest business risk.

Lessons Learned

This review reinforces several important observations regarding AI-assisted software testing.

1. AI is highly effective at generating an initial regression suite from a written specification.
2. Human review is essential to remove hallucinated behaviour and implementation assumptions.
3. AI performs well with structured testing techniques but less effectively with exploratory and risk-based thinking.
4. Requirement ambiguities are often more valuable than additional boundary tests and should always be identified by the reviewer.
5. The highest-quality test suites are produced through collaboration between AI and experienced QA professionals.

Conclusion

The generated test suite provides an excellent foundation for validating the Digital Wallet Transfer Limits feature. The AI successfully produced extensive functional coverage and demonstrated strong application of established test design techniques such as Boundary Value Analysis and Equivalence Partitioning.

However, the review also confirms that AI-generated artefacts cannot be accepted without critical evaluation. Several test cases contain hallucinated implementation details, unnecessary duplication, and assumptions that are unsupported by the specification. In addition, important business risks and requirement ambiguities remain unaddressed.

Approximately 75-80% of the generated test cases should be retained with minor refinement, 10-15% should be rewritten to remove implementation-specific assumptions, and 10-15% should be consolidated or removed due to duplication. Additional manually designed scenarios should be introduced to address concurrency, transaction reversals, idempotency, state transitions, and unresolved business ambiguities.

In conclusion, this exercise demonstrates that AI is most effective when used as a productivity tool that accelerates test design, while the expertise of a QA engineer remains essential to validate requirements, challenge assumptions, prioritise business risk, and produce a complete and reliable test suite suitable for production use.